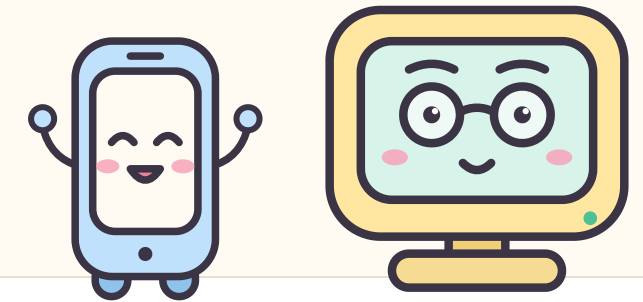




第 21 集 · 约 38 分钟

加餐：新设备 要电脑点头

二维码泄露了，也连不上来

配套阅读：书的第 10 章

21

从零写出 LanShare

这一集你会学到

1. 设计安全功能，先想清楚“记住谁”、“门放在哪”、挡得住什么、挡不住什么
2. 带数据的枚举和 matches! 宏
3. 把“现在几点”当参数传进来，测时限不用真的等
4. 锁里只算，出了锁再做慢事
5. 在测试齐全的项目上加功能：只加新行，不改旧代码
6. 顺手补一个洞：开新会话也要限速



一个真实的问题

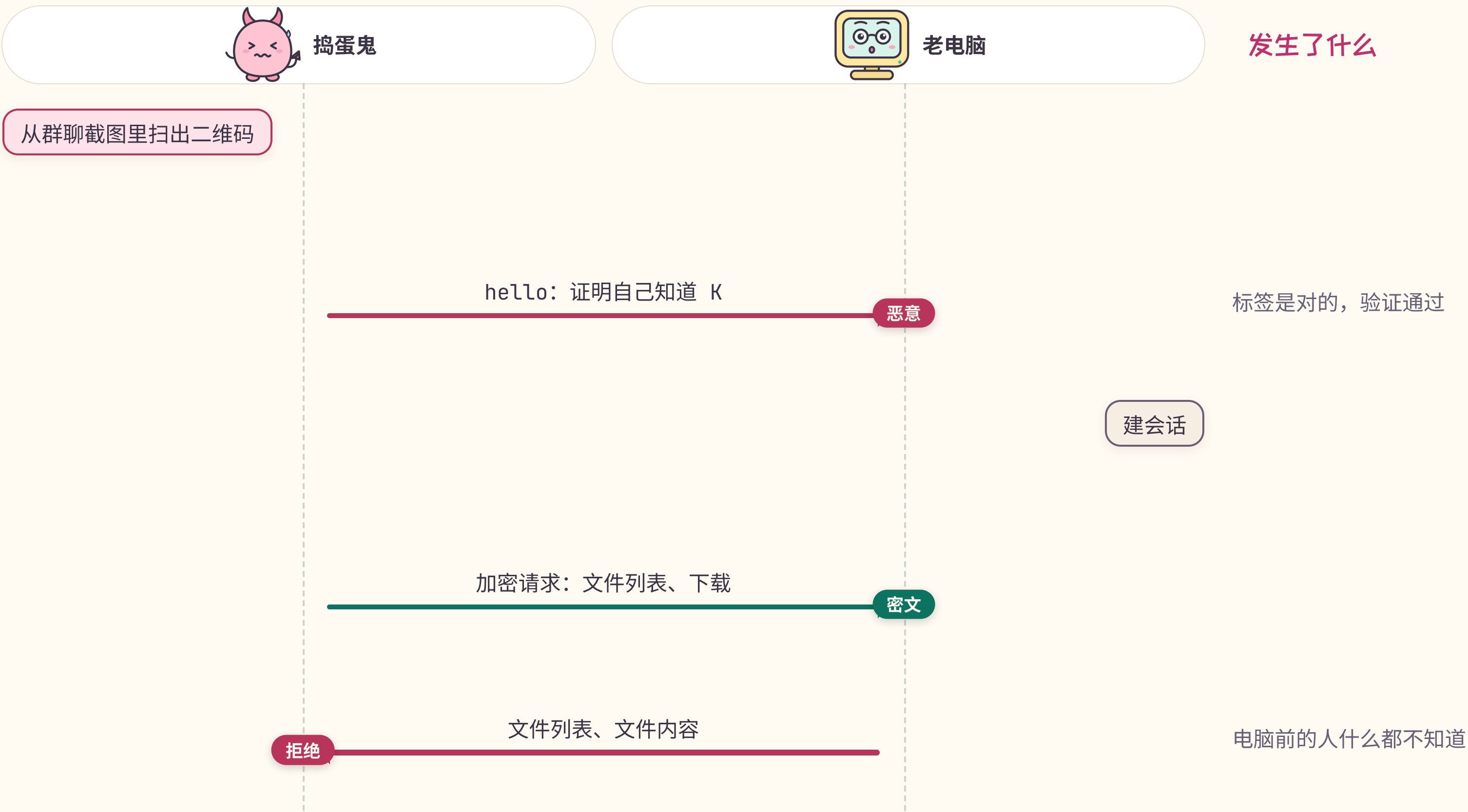
二维码泄露了，是不是就完蛋了？

以前是：拿到二维码的人，就是可信用户。这一集给老电脑加一道门。



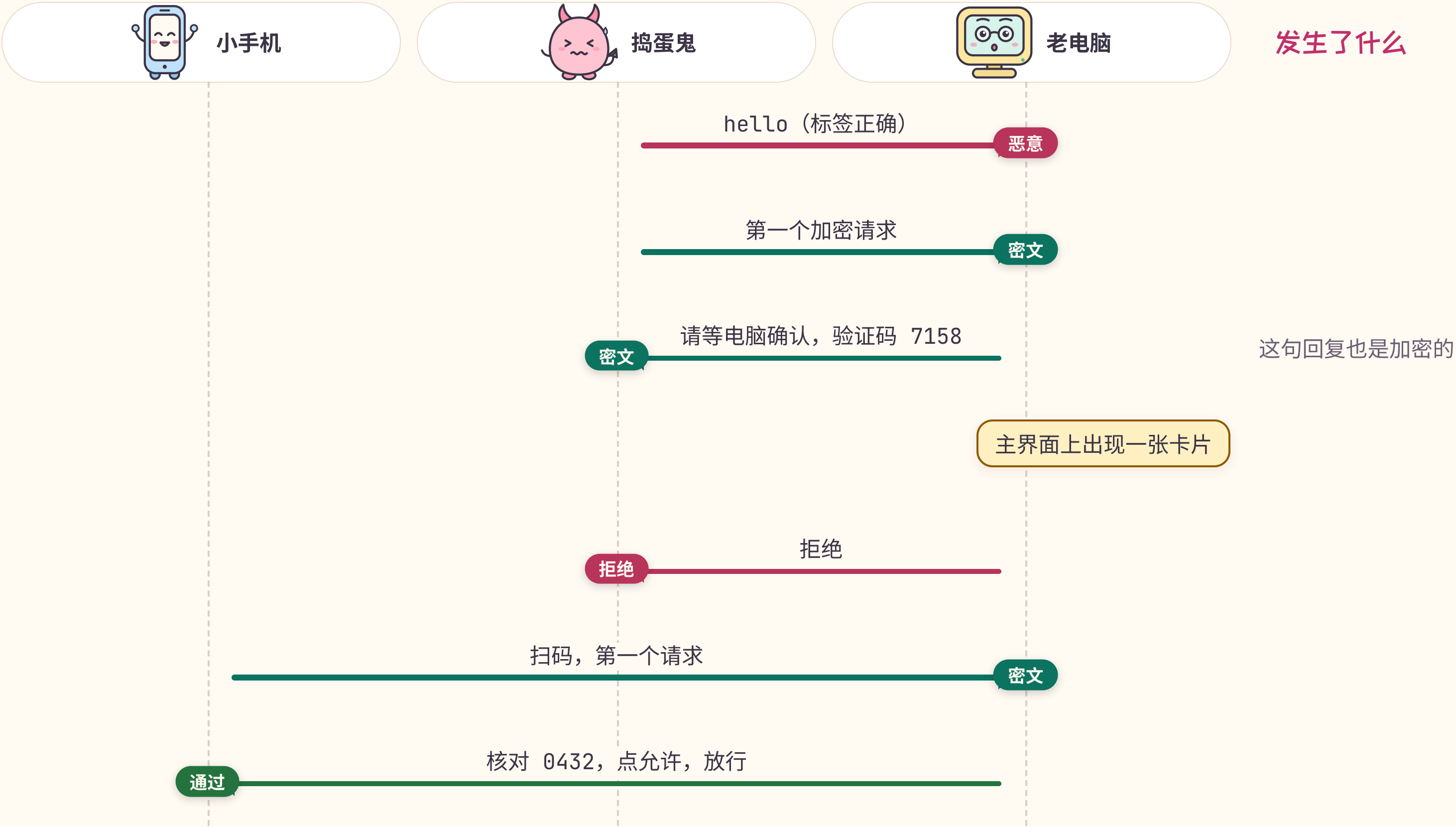


以前：捣蛋鬼捡到了二维码





加了门以后





记住谁：三个候选

点了“允许”以后，要记住点什么，下次才不用再点

- 按会话记

手机刷新一次页面就换一个会话，就要再点一次

- 给设备发一个令牌

令牌只能明文放在握手里，或者放在用 K 派生的信道里

- 按 IP 记，这次运行内有效

程序重启就清空，和 K、口令同一个寿命



门放在哪

门放的位置，决定了要改多少东西

- 放在握手里
 - 要同时改协议核心、WASM、proto.js 和已知向量
- 放在加密信道里：解密成功之后、交给 RPC 之前
 - 握手和协议一个字节都不用动
- 能走到门口的，一定通过了握手
 - 不知道 K 也不知道口令的人，连“待确认”都登记不上



挡得住什么，挡不住什么

情况	结果
拿到二维码或口令，在别的设备上连接	握手能成功，但什么都拿不到；电脑上冒出一条陌生设备，点拒绝
猜中了口令	同上：猜中了也要电脑上点允许
拿到二维码，又在同一个 WiFi 里抓包	挡不住：能解密已经放行的设备的流量
冒用已经放行的设备的 IP	挡不住：门按 IP 记；冒用 IP 属于主动攻击，本来就不在防护范围
拿到二维码，反复连接捣乱	占满 8 个待确认名额、每 30 秒弹一次窗、反复握手挤掉别人的会话：会打扰，不会泄密
自己点错了允许	从托盘退出程序再打开：K、口令、放行记录一起作废



只加新行，不改旧代码

在测试齐全的项目上加功能，最稳的办法

● 所有改动都是新增的行

做完以后，git diff 里看不到一个减号

● 门有一个开关：正式运行默认打开

测试工具默认关掉，只有新功能自己的测试打开

● 网页只用已有的样式

style.css 一行不动



approval.rs: 开头

● ● ●

approval.rs crates/lanshare/src/server/

第 29–48 行 • 共 48 行

```
29     /// 待确认的设备太多了。
30     Busy,
31 }
32
33 struct Waiting {
34     id: String,
35     ip: IpAddr,
36     device: &'static str,
37     code: String,
38     last_seen: Instant,
39 }
40
41 #[derive(Default)]
42 pub(crate) struct Approvals {
43     allowed: HashSet<IpAddr>,
44     rejected: HashMap<IpAddr, Instant>,
45     waiting: Vec<Waiting>,
46     desktop_seen: Option<Instant>,
47     popped: Option<Instant>,
48 }
```



approval.rs: 门本身

approval.rs crates/lanshare/src/server/

第 101-120 行 • 共 120 行

```

101         self.rejected.insert(w.ip, now);
102     }
103     Some((w.ip, w.device))
104 }
105 }
106
107 /// 从 User-Agent 粗分设备类型，只用来帮电脑前的人认出是哪台设备。User-Agent 可以随便填，不能当身份。
108 pub(crate) fn device_label(user_agent: &str) -> &'static str {
109     // 顺序有讲究：安卓的 User-Agent 里也有 Linux, iPhone 的里也有 Mac OS X
110     const KINDS: [(&str, &str); 7] = [
111         ("iPhone", "iPhone"),
112         ("iPad", "iPad"),
113         ("Android", "安卓设备"),
114         ("Windows", "Windows 电脑"),
115         ("Macintosh", "Mac"),
116         ("CrOS", "Chromebook"),
117         ("Linux", "Linux 电脑"),
118     ];
119     KINDS.iter().find(|(needle, _)| user_agent.contains(needle)).map_or("未知设备", |(_, label)| label)
120 }

```



approval.rs: 用假时间测时限

approval.rs

crates/lanshare/src/server/

第 192-211 行 • 共 211 行

```

192     let t = Instant::now();
193     assert!(a.should_pop(t), "主界面从未来问过：弹出来");
194     assert!(!a.should_pop(t + Duration::from_secs(5)), "刚弹过，不再弹");
195     let t2 = t + POPUP_EVERY + Duration::from_secs(1);
196     a.waiting(t2);
197     assert!(!a.should_pop(t2 + Duration::from_secs(1)), "主界面开着：不弹");
198     assert!(a.should_pop(t2 + DESKTOP_AWAY + Duration::from_secs(1)), "主界面关了一阵：再弹");
199 }

200
201 #[test]
202 fn device_labels() {
203     let iphone = "Mozilla/5.0 (iPhone; CPU iPhone OS 17_5 like Mac OS X) AppleWebKit/605.1.15 Safari/604.1";
204     let android = "Mozilla/5.0 (Linux; Android 14; Pixel 8) AppleWebKit/537.36 Chrome/126.0 Mobile Safari/537.36";
205     let windows = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Chrome/126.0 Safari/537.36 Edg/126.0";
206     assert_eq!(device_label(iphone), "iPhone");
207     assert_eq!(device_label(android), "安卓设备");
208     assert_eq!(device_label(windows), "Windows 电脑");
209     assert_eq!(device_label(""), "未知设备");
210 }
211 }

```



mod.rs: 把门装进服务端

● ● ●

mod.rs

crates/lanshare/src/server/

第 209-228 行 • 共 453 行

```
209     approvals: Mutex::new(approval::Approvals::default()),
210     approve_new_devices: config.approve_new_devices,
211     files: Mutex::new(FileIds::default()),
212     uploads: Mutex::new(transfer::Uploads::default()),
213     desktop: config.desktop,
214     log: config.log,
215     max_sessions: config.max_sessions,
216     per_ip_limit: config.per_ip_limit,
217     global_limit: config.global_limit,
218     pin_fail_limit: config.pin_fail_limit,
219     max_uploads: config.max_uploads,
220     max_uploads_per_session: config.max_uploads_per_session,
221     upload_idle: config.upload_idle,
222     disk_reserve: config.disk_reserve,
223     body_timeout: config.body_timeout,
224     idle_timeout: config.idle_timeout,
225     max_connections: config.max_connections,
226     max_local_connections: config.max_local_connections,
227     max_connections_per_ip: config.max_connections_per_ip,
228     header_timeout: config.header_timeout,
```



shell.rs: 真的弹出主界面

● ● ●

shell.rs crates/lanshare/src/desktop/

第 141–160 行 • 共 186 行

```
141     }
142     fn open_app(&self, url: &str) {
143         open_app_window(url);
144     }
145 }
146
147 #[cfg(test)]
148 mod tests {
149     use super::*;
150
151     #[test]
152     fn strips_verbatim_prefixes() {
153         assert_eq!(display_path(Path::new(r"\\?\C:\Users\a\Downloads\LanShare\x.mp4")), PathBuf::from(r"C:\Users\a\Downloads\LanShare\x.mp4"));
154         assert_eq!(display_path(Path::new(r"\\?\UNC\nas\share\x.mp4")), PathBuf::from(r"\\nas\share\x.mp4"));
155         assert_eq!(display_path(Path::new(r"C:\plain\x.mp4")), PathBuf::from(r"C:\plain\x.mp4"));
156     }
157
158     #[test]
159     fn explorer_argument_keeps_spaces_and_commas_inside_quotes() {
160         let arg = explorer_select_arg(Path::new(r"\\?\C:\共享 文件夹\报价, 终版.pdf"));
```




channel.rs: 过门



channel.rs

crates/lanshare/src/server/

第 164–183 行 • 共 183 行

```
164     let opened = match open_body(&st, cid, ctr, &keys, &path, &body) {
165         Ok(opened) => opened,
166         Err(status) => return status.into_response(),
167     };
168     let local = peer.ip().is_loopback();
169     // 局域网里的新设备：电脑上点了“允许”才放行。分块上传下载直接 403：没放行的设备还拿不到文件列表和上传号
170     if !local && st.approve_new_devices && let Some(reply) = gate(&st, peer.ip(), &parts.headers) {
171         return if path == "/api/rpc" { sealed_response(&opened, &path, &reply) } else { StatusCode::FORBIDDEN.into_response() };
172     }
173     let segments: Vec<&str> = path.trim_start_matches('/').split('/').collect();
174     let payload: Vec<u8> = match segments.as_slice() {
175         ["api", "rpc"] => rpc::handle(&st, &opened.plain, local, opened.cid, peer.ip()).await,
176         ["api", "up", upload, index] => {
177             transfer::put_chunk(&st, upload, index, opened.plain.clone(), opened.cid).await
178         }
179         ["api", "down", file, index] => transfer::get_chunk(&st, file, index, &opened.plain, peer.ip()).await,
180         _ => rpc::reply_error("not_found"),
181     };
182     sealed_response(&opened, &path, &payload)
183 }
```



rpc.rs: 电脑上的三个操作

rpc.rs crates/lanshare/src/server/

第 122-141 行 • 共 145 行

```
122 /// 本机主界面上点“允许”或“拒绝”。只认本机：新设备不能自己放行自己。
123 fn decide(st: &AppState, request: &Value, local: bool, allow: bool) -> Vec<u8> {
124     if !local {
125         return reply_error("only_local");
126     }
127     let id = request["id"].as_str().unwrap_or("");
128     let Some((ip, device)) = st.approvals.lock().expect("锁不会中毒").decide(id, allow, Instant::now()) else {
129         return reply_error("not_found");
130     };
131     st.log(&format!("{ip}: {device} ", if allow { "已允许新设备" } else { "已拒绝新设备" }));
132     reply(json!({ "ok": true }))
133 }
134
135 fn open_folder(st: &AppState, local: bool) -> Vec<u8> {
136     if !local {
137         return reply_error("only_local");
138     }
139     let Some(desktop) = st.desktop.clone() else { return reply_error("unavailable") };
140     if st.storage.ensure_root().is_err() {
141         return reply_error("folder_unavailable");
```



测试工具：假桌面和开关

● ● ●

mod.rs

crates/lanshare/tests/common/

第 63–82 行 • 共 244 行

```
63     config.approve_new_devices = false; // 新设备确认只在 tests/approval.rs 里打开，别的测试各测各的
64     tweak(&mut config);
65     let state = Arc::new(AppState::new(config, port));
66     tokio::spawn(server::serve(listener, state.clone()));
67     TestServer { base: format!("http://127.0.0.1:{port}"), port, state, dir, desktop, logs }
68 }
69
70 pub async fn start() -> TestServer {
71     start_with(Ipv4Addr::LOCALHOST, |_| {}).await
72 }
73
74 pub fn http() -> reqwest::Client {
75     reqwest::Client::builder().no_proxy().build().unwrap()
76 }
77
78 pub fn random_cid() -> [u8; 16] {
79     let mut cid = [0u8; 16];
80     OsRng.fill_bytes(&mut cid);
81     cid
82 }
```




tests/approval.rs: 拷贝过来，挑三段讲

approval.rs

crates/lanshare/tests/

从仓库拷贝

第 124-143 行 • 共 148 行

```

124
125 #[tokio::test]
126 async fn desktop_pops_up_only_when_nobody_is_watching() {
127     let (s, lan_ip, mut phone) = need_lan!(lan_phone().await);
128     phone.rpc(json!({ "op": "info" })).await;
129     let calls = s.desktop.calls.lock().unwrap().clone();
130     assert_eq!(calls.len(), 1, "主界面没开: 弹出来");
131     assert_eq!(calls[0].0, "app");
132     assert_eq!(calls[0].1.to_str().unwrap(), s.state.local_entry_url());
133
134     // 主界面开着 (刚来问过) : 不弹
135     let quiet = start_with(Ipv4Addr::UNSPECIFIED, |c| c.approve_new_devices = true).await;
136     pair_qr(&quiet).await.rpc(json!({ "op": "approvals" })).await;
137     scan(&quiet, lan_ip).await.rpc(json!({ "op": "info" })).await;
138     assert!(quiet.desktop.calls.lock().unwrap().is_empty());
139 }
140
141 #[tokio::test]
142 async fn loopback_needs_no_approval() {
143     let s = start_with(Ipv4Addr::LOCALHOST, |c| c.approve_new_devices = true).await;

```



跑新功能的测试

```
Windows PowerShell

PS D:\code\lanshare> cargo test -p lanshare --lib approval
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
  Finished `test` profile [unoptimized + debuginfo] target(s) in 10.61s
  Running unittests src\lib.rs (target\debug\deps\lanshare-152f780c5f39ffeb.exe)

running 6 tests
test server::approval::tests::device_labels ... ok
test server::approval::tests::pops_up_only_when_desktop_is_away_and_not_too_often ... ok
test server::approval::tests::idle_requests_expire_and_the_list_is_bounded ... ok
test server::approval::tests::new_device_waits_with_a_stable_code ... ok
test server::approval::tests::allow_lets_every_session_from_that_ip_through ... ok
test server::approval::tests::reject_holds_for_a_while_then_can_ask_again ... ok

test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 36 filtered out; finished in 0.00s

PS D:\code\lanshare> cargo test -p lanshare --test approval
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Finished `test` profile [unoptimized + debuginfo] target(s) in 8.82s
  Running tests\approval.rs (target\debug\deps\approval-1ee28a433511e223.exe)
```



proto.js: 记住电脑发来的 K

proto.js crates/lanshare/web/

第 197-216 行 • 共 368 行

```
197  /** 记住电脑经加密信道发来的 K (`info` 里的 key)。用口令配对的设备，要等电脑上点了“允许”才拿得到。 */
198  rememberKey(fragment) {
199      const key = keyFromFragment("#" + fragment);
200      if (key) this.#remember(key);
201  }
202
203  #useSession(cid, sessionSecret) {
204      this.cid = cid;
205      this.keys = this.crypto.channelKeys(sessionSecret);
206      this.ctr = 0;
207  }
208
209  get paired() {
210      return this.keys !== null;
211  }
212
213  /** 扫码配对：证明知道二维码里的密钥 K，派生本会话的密钥。 */
214  async pairWithKey(key) {
215      await this.serverId();
216      const cid = this.crypto.randomBytes(16);
```



index.html: 两块新界面

index.html crates/lanshare/web/

第 62-81 行 · 共 130 行

```
62     <section class="card" id="approvals" hidden>
63         <div class="card-head"><h2>有新设备想连接</h2></div>
64         <p class="muted">先核对验证码：和那台设备屏幕上的一样，再点“允许”。不认识的设备，点“拒绝”。</p>
65         <ul id="approvalList" class="files"></ul>
66     </section>
67
68     <section class="card" id="upload">
69         <label id="drop" class="drop">
70             <input id="picker" class="visually-hidden" type="file" multiple>
71             <span class="drop-icon"><svg class="i"><use href="#i-upload"/></svg></span>
72             <span>
73                 <span class="drop-title" id="dropTitle">选择文件发送</span><br>
74                 <span class="muted" id="dropHint">可以多选，照片、视频、文档都行</span>
75             </span>
76         </label>
77         <ul id="queue" class="uploads"></ul>
78     </section>
79
80     <section class="card" id="files">
81         <div class="card-head">
```



app.js: 手机上等一等



app.js

crates/lanshare/web/

第 167-186 行 • 共 524 行

```
167 client.rememberKey(info.key);
168 isDesktop = !!info.local;
169 document.body.classList.toggle("desktop", isDesktop);
170 document.body.classList.toggle("mobile", !isDesktop);
171 const main = info.urls[0].replace(/\$/ , "");
172 $("url").textContent = main;
173 showPin(info);
174 const others = info.urls.slice(1).map((u) => u.replace(/\$/ , ""));
175 $("others").hidden = others.length === 0;
176 $("others").textContent = "扫码地址打不开时，在手机浏览器里试试：" + others.join("、");
177 if (qrUrl) URL.revokeObjectURL(qrUrl);
178 qrUrl = URL.createObjectURL(new Blob([info.qr_svg], { type: "image/svg+xml" }));
179 $("qr").src = qrUrl;
180 $("connectFold").open = isDesktop;
181 $("connectTitle").textContent = isDesktop ? "用手机扫码连接" : "让其他设备也连上来";
182 $("openFolder").hidden = !isDesktop;
183 $("trayNote").hidden = !isDesktop; // v1 用启动气泡提示这句话；v2 的托盘库没有气泡，改成写在本机界面上
184 $("chipText").textContent = isDesktop ? "运行中 · " + main.replace("http://", "") : "已连接";
185 $("dropTitle").textContent = isDesktop ? "把文件拖到窗口里，或点这里选择" : "选择文件发送";
186 $("dropHint").textContent = isDesktop ? "所有连上的手机、电脑都能下载" : "可以多选，照片、视频、文档都行";
```




app.js: 电脑上点头

app.js crates/lanshare/web/

第 292-311 行 • 共 569 行

```
292
293 async function decide(device, allow) {
294   try {
295     const r = await client.rpc({ op: allow ? "approve" : "reject", id: device.id });
296     toast(r.ok ? (allow ? "已允许 " : "已拒绝 ") + device.device : "这台设备已经不在等了");
297   } catch {
298     toast("操作失败，请重试");
299   }
300   refresh();
301 }
302
303 // ----- 文件列表 -----
304
305 function renderFiles(files) {
306   const key = JSON.stringify(files) + isDesktop;
307   if (key === lastFilesKey) return;
308   lastFilesKey = key;
309   $("fileCount").textContent = files.length ? files.length + " 个" : "";
310   const list = $("fileList");
311   list.textContent = "";
```



page.test.mjs: 元素都在吗

page.test.mjs web-tests/

第 18-37 行 · 共 52 行

```

18 test("app.js 里用到的元素，页面上都有", () => {
19     const ids = [...scripts["app.js"].matchAll(/\$\\("([A-Za-z]+)"\\)/g)].map((m) => m[1]);
20     assert.ok(ids.length > 20);
21     for (const id of new Set(ids)) assert.ok(html.includes(`id="${id}"`), id);
22 });
23
24 test("没有内联脚本、内联样式、内联事件 (CSP 会拦掉)", () => {
25     const scriptTags = html.match(/<script\b[^\>]*>[\s\S]*?<\/script>/g) || [];
26     for (const tag of scriptTags) {
27         assert.match(tag, /\bsrc="\/[a-z]+\s*.js"/, "脚本必须外链本站文件");
28         assert.match(tag, /><\/script>$/, "script 标签里不能有内容");
29     }
30     assert.doesNotMatch(html, /\sstyle\s*/i, "不能有 style 属性");
31     assert.doesNotMatch(html, /\son[a-z]+\s*/i, "不能有 onclick 之类的内联事件");
32     assert.doesNotMatch(html, /<style\b/i, "不能有 <style> 块");
33 });
34
35 test("不引用任何外部资源 (断网局域网也要能用)", () => {
36     for (const [name, text] of [
37         ["index.html", html],
38         ["style.css", css],
39         ...Object.entries(scripts)
40     ]) {
41         assert.doesNotMatch(text, /(src|href)\s*=\s*"([^"]*)"?(https?:)?\s*\/\s*\//i, name);
42     }
43 }

```



小手机上：等电脑点头



第一次连上来的手机（测试时的截图）



老电脑上：有新设备想连接



电脑上的主界面（截图里的地址、口令、验证码都是测试时的值）





门挡住了，他还能干什么

捡到二维码的人看不到文件了，可他手里还有 K

- 他还能不停地扫码握手

握手只限制失败的次数；标签是对的，就一直成功

- 会话表满了，每开一个就挤掉一个旧的

被挤掉的会话号要记进 retired，这次运行内一直记着

- 补法：开新会话也要限速

每个 IP 每分钟 30 个，全局 300 个；同一个会话号再握手不算



auth.rs: 开新会话也要限速

auth.rs crates/lanshare/src/server/

第 337-356 行 • 共 359 行

```
337
338 #[cfg(test)]
339 mod tests {
340     use super::*;
341
342     #[test]
343     fn new_sessions_are_counted_per_minute() {
344         let mut guard = Guard::default();
345         let ip = IpAddr::from([192, 168, 1, 7]);
346         let t = Instant::now();
347         assert!(guard.take_new_session(ip, t, 2, 100));
348         assert!(guard.take_new_session(ip, t, 2, 100));
349         assert!(!guard.take_new_session(ip, t, 2, 100), "这一分钟里的第 3 个");
350         assert!(guard.take_new_session(IpAddr::from([192, 168, 1, 8]), t, 2, 100), "别的 IP 不受影响");
351         assert!(guard.take_new_session(ip, t + WINDOW + Duration::from_secs(1), 2, 100), "过了一分钟又可以");
352
353         let mut guard = Guard::default();
354         for last in 1..=3 {
355             assert!(guard.take_new_session(IpAddr::from([192, 168, 1, last]), t, 100, 3));
356         }
357     }
358 }
```



mod.rs: 两个新的上限

● ● ●

mod.rs

crates/lanshare/src/server/

第 225-244 行 • 共 462 行

```
225     new_sessions_per_ip: config.new_sessions_per_ip,
226     new_sessions_global: config.new_sessions_global,
227     pin_fail_limit: config.pin_fail_limit,
228     max_uploads: config.max_uploads,
229     max_uploads_per_session: config.max_uploads_per_session,
230     upload_idle: config.upload_idle,
231     disk_reserve: config.disk_reserve,
232     body_timeout: config.body_timeout,
233     idle_timeout: config.idle_timeout,
234     max_connections: config.max_connections,
235     max_local_connections: config.max_local_connections,
236     max_connections_per_ip: config.max_connections_per_ip,
237     header_timeout: config.header_timeout,
238     handshake_timeout: config.handshake_timeout,
239     bodies: Arc::new(tokio::sync::Semaphore::new(config.max_inflight_bodies)),
240     max_bodies_per_ip: config.max_bodies_per_ip,
241     bodies_per_ip: Mutex::new(HashMap::new()),
242     alloc_lock: Mutex::new(()),
243 }
244 }
```



tests/handshake_limits.rs: 拷贝过来，挑两段讲

● ● ●

handshake_limits.rs

crates/lanshare/tests/

从仓库拷贝

第 44-62 行 · 共 62 行

```
44
45 #[tokio::test]
46 async fn refused_handshakes_do_not_push_out_live_sessions() {
47     let s = start_with(Ipv4Addr::LOCALHOST, |c| {
48         c.max_sessions = 2;
49         c.new_sessions_per_ip = 2;
50     })
51     .await;
52     let mut a = pair_qr(&s).await;
53     let mut b = pair_qr(&s).await;
54     for _ in 0..5 {
55         let cid = random_cid();
56         assert_eq!(hello(&s.base, &s.key(), &cid).await.status(), 429);
57         let mut ghost = Client::from_secret(&s.base, cid, &proto::session_secret_from_key(&s.key(), &cid));
58         assert_eq!(ghost.send("/api/rpc", br#"{"op":"info"}"#).await.err(), Some(401), "被拒的握手没有建会话");
59     }
60     assert_eq!(a.rpc(json!({ "op": "info" }))).await["ok"], true, "会话表满了，被拒的握手也挤不掉已有的会话");
61     assert_eq!(b.rpc(json!({ "op": "info" }))).await["ok"], true);
62 }
```




跑限速的测试

```
Windows PowerShell

PS D:\code\lanshare> cargo test -p lanshare --lib auth
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.42s
    Running unittests src\lib.rs (target\debug\deps\lanshare-152f780c5f39ffeb.exe)

running 1 test
test server::auth::tests::new_sessions_are_counted_per_minute ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 41 filtered out; finished in 0.00s

PS D:\code\lanshare> cargo test -p lanshare --test handshake_limits
    Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 1.96s
    Running tests\handshake_limits.rs (target\debug\deps\handshake_limits-3207c54e9c644ec8.exe)

running 4 tests
test limits_by_default ... ok
test new_sessions_are_limited_globally ... ok
test new_sessions_per_ip_are_limited ... ok
test refused_handshakes_do_not_push_out_live_sessions ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.05s
```





全部测试，重新打包

```
Windows PowerShell

PS D:\code\lanshare> cargo build -p lanshare
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 6.43s

PS D:\code\lanshare> node --test "web-tests/*.test.mjs"
WASM seal 355 MiB/s, open 370 MiB/s (Node v24.12.0, 1 MiB chunks)
..... (省略 18 行)
✓ 页面有全部关键元素 (0.7853ms)
✓ app.js 里用到的元素，页面上都有 (0.3155ms)
..... (省略 4 行)
i tests 24
..... (省略 1 行)
i pass 24
i fail 0
..... (省略 4 行)

PS D:\code\lanshare> powershell -NoProfile -ExecutionPolicy Bypass -File build.ps1
==> build wasm
..... (省略 2 行)
  Finished `release` profile [optimized] target(s) in 1.81s
==> clippy
..... (省略 3 行)
```



真实踩坑

● 改了一行 use，“只加不改”就破了

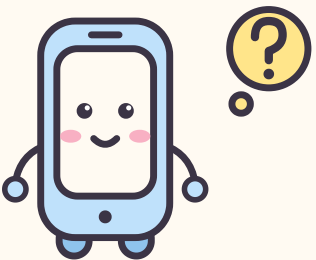
照着文件里已有的写法，用完整路径 `std::net::IpAddr`

● 第一版改了两个旧测试，第二版一个都不改

新功能的开关，正式运行默认开，测试里按需开

● 断言要对准你想证明的那件事

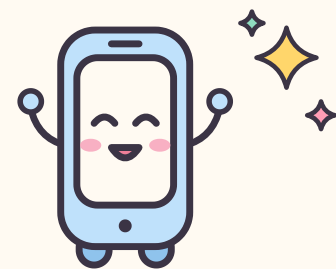
“本机不会登记成新设备”，不等于“日志里没有 127.0.0.1”



选择题

为什么不按会话记“允许过”？

- A 会话号是明文的，不安全
- B 手机每次打开或刷新页面都会重新握手、换新会话，按会话记就要反复在电脑上点
- C 会话太多，内存放不下



选择题

为什么不按会话记“允许过”？

A 会话号是明文的，不安全

会话号本来就是明文的，这和记不记住无关。

C 会话太多，内存放不下

会话表本来就有上限，这不是原因。

B 手机每次打开或刷新页面都会重新握手、换新会话，按会话记就要反复在电脑上点

手机用存下的 K 重新握手，每次都换一个新的会话号。按 IP 记，这次运行内同一台设备就不用再点了。



选择题

下面哪种请求，能在电脑上登记成一台“待确认”的设备？

- A 任何一个知道电脑地址的人发来的请求
- B 握手成功、能正确加密的局域网设备发来的请求
- C 本机主界面发来的请求



选择题

下面哪种请求，能在电脑上登记成一台“待确认”的设备？

A 任何一个知道电脑地址的人发来的请求

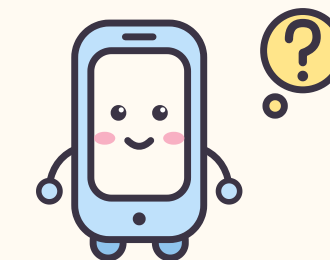
门在加密信道里，解密不了的请求根本走不到门口。

C 本机主界面发来的请求

本机来的请求不过门，本机主界面就是点允许的地方。

B 握手成功、能正确加密的局域网设备发来的请求

门在解密成功之后。没有 K 也没有口令的人，连待确认都登记不上，没法往电脑上刷假请求。



选择题

拿到二维码、又能在同一个 WiFi 里抓包的人，这道门挡得住吗？

- A 挡得住，因为他的设备要电脑上点允许
- B 挡不住：他能解密已经放行的设备的流量，因为会话密钥由 K 和明文的会话号派生
- C 挡得住，因为验证码是加密传输的



选择题

拿到二维码、又能在同一个 WiFi 里抓包的人，这道门挡得住吗？

A 挡得住，因为他的设备要电脑上点允许

他自己的设备确实会被挡住，但他还有别的办法。

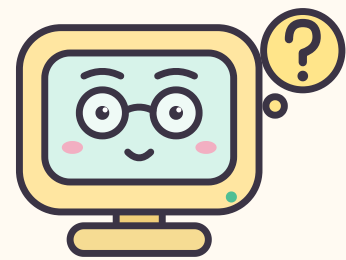
C 挡得住，因为验证码是加密传输的

验证码加密只保证别人看不到验证码，挡不住用 K 算出别人的会话密钥。

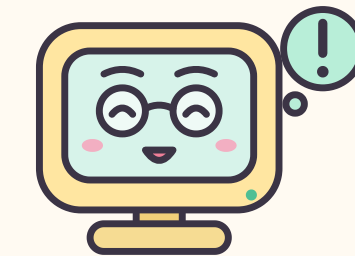
B 挡不住：他能解密已经放行的设备的流量，因为会话密钥由 K 和明文的会话号派生

持有 K 又看得到会话号，就能算出已经放行的设备的会话密钥，看到它传了什么。他想用这个会话发请求，还得冒用那台设备的 IP。要挡住，得在握手里加一次性的密钥交换，这一集没做，安全说明里照实写了。

想一想



如果能让“点错了允许”不用重启程序就能撤销，你会怎么加？要改哪些地方？



想一想

如果能让“点错了允许”不用重启程序就能撤销，你会怎么加？要改哪些地方？

在 Approvals 里加一个撤销方法，把 IP 从允许集合里删掉；RPC 加一个只认本机的撤销操作；主界面列出已经放行的设备，每条一个撤销按钮。因为门每个请求都会问一次，撤销以后，这个 IP 的下一个请求就会重新变成待确认，不用去动会话表。测试要覆盖：撤销以后旧会话马上被挡住、只有本机能撤销、撤销不存在的设备回 not_found。



这一集写了什么

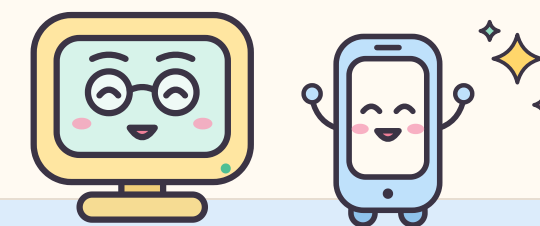
```
lanshare/  
├── crates/  
│   └── lanshare/  
│       ├── src/  
│       │   ├── desktop/  
│       │   │   ├── instance.rs  
│       │   │   ├── mod.rs  
│       │   │   ├── shell.rs  这一集写的  
│       │   │   └── tray.rs  
│       │   └── server/  
│       │       ├── approval.rs  这一集写的  
│       │       ├── assets.rs  
│       │       ├── auth.rs  这一集写的  
│       │       ├── channel.rs  这一集写的  
│       │       ├── conn.rs  
│       │       ├── mod.rs  这一集写的  
│       │       ├── rpc.rs  这一集写的  
│       │       └── transfer.rs  
│       ├── cli.rs  
│       ├── lib.rs  
│       ├── logging.rs  
│       ├── main.rs  
│       ├── net.rs  
│       └── netinfo.rs
```

```
storage.rs  
textboard.rs  
tests/  
├── common/  
│   └── mod.rs  这一集写的  
├── api.rs  从仓库拷贝  
├── approval.rs  这一集写的  
├── handshake_limits.rs  这一集写的  
├── hardening.rs  从仓库拷贝  
├── transfer.rs  从仓库拷贝  
└── wire.rs  从仓库拷贝  
web/  
├── app.js  这一集写的  
├── icon.ico  从仓库拷贝  
├── index.html  这一集写的  
├── lanshare.wasm  从仓库拷贝  
├── proto.js  这一集写的  
└── style.css  从仓库拷贝  
build.rs  
Cargo.toml  
lanshare-proto/ 13 个文件  
lanshare-wasm/ 2 个文件  
web-tests/  
└── crypto.test.mjs
```



这一集我们学了

- ✓ 先想清楚“记住谁”：按 IP 记，这次运行内有效
- ✓ 门放在加密信道里：解密成功之后、交给 RPC 之前，握手和协议都不动
- ✓ 带数据的枚举和 matches!；把“现在几点”当参数传进来
- ✓ 锁里只算，出了锁再做慢事
- ✓ 只加不改：新功能的开关，正式运行默认开，测试里按需开
- ✓ 顺手补洞：开新会话也要限速，让只增不减的退役表涨不快
- ✓ 照实写边界：持有 K 又能抓包的人能解密，冒用 IP 的主动攻击也挡不住



下一集

全课完结，感谢收看